Facade Pattern

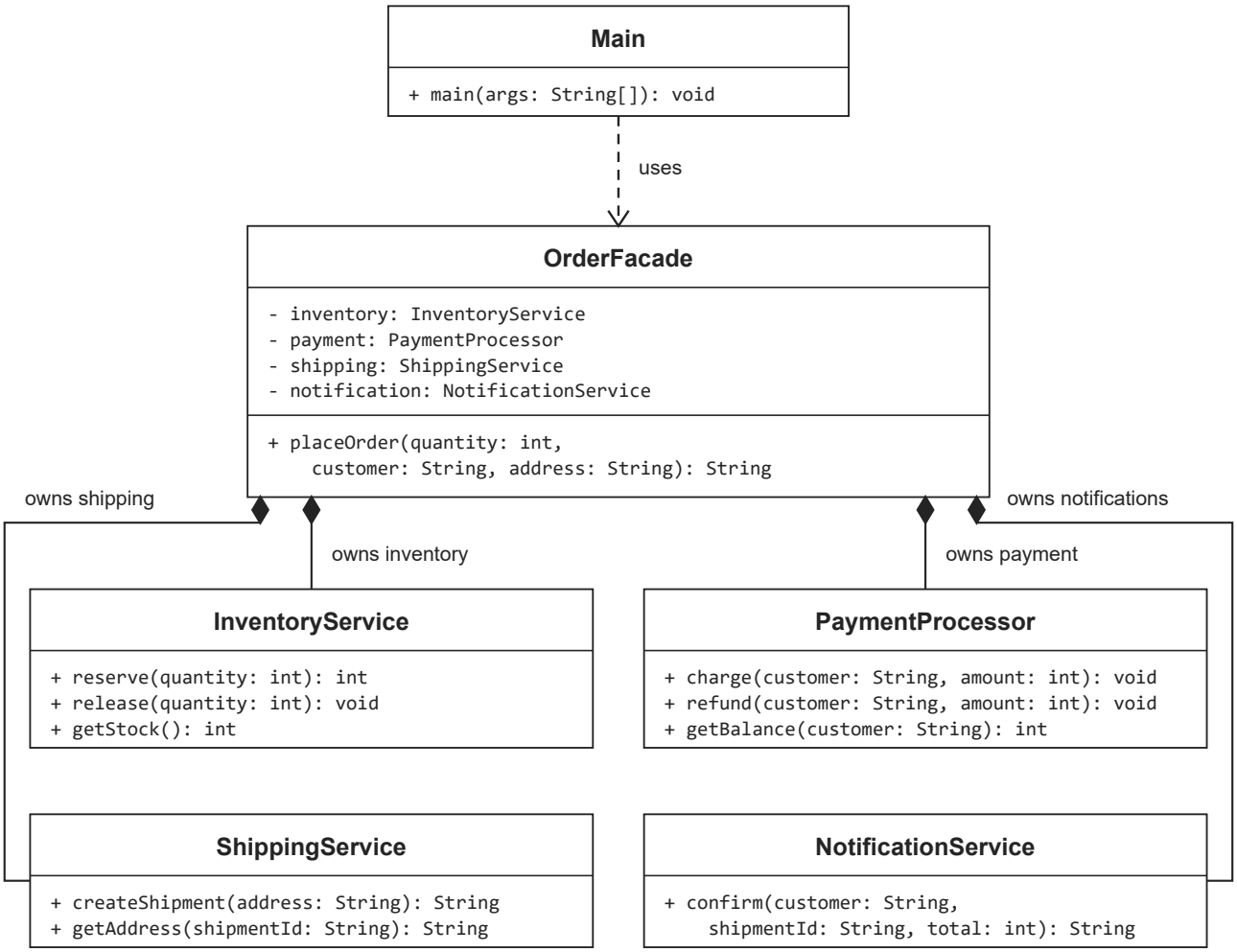
Assignment 4 · Online Order Processing
ShP-2216 · Software Design Patterns · Java 17

1 Introduction

The program processes orders for Java Book, which costs 1000 KZT. It reserves stock, charges the customer, creates a shipment and stores a notification. Without a facade, the client would have to call four services in the correct order and handle failures between these steps.

OrderFacade combines these steps into one method, placeOrder. Main provides the quantity, customer name and address. It receives a confirmation or catches an exception and displays the error message.

2 UML Class Diagram



The dashed arrow labeled uses is a dependency. Each owns relationship is a composition, shown by a filled diamond at OrderFacade. The facade creates and holds one instance of each service. + means public; - means private. The diagram shows the main operations.

Workflow

Inventory → payment → shipping → notification. If payment fails, the facade releases the reserved books. If the address is invalid after payment, it also refunds the customer.

3 Clean Code Principles

The annotations below refer to the actual VS Code screenshots on pages 4-9. Each screenshot shows one complete source file, including its line numbers. The source is shown as images rather than retyped excerpts.

1 Meaningful Names

The names placeOrder, quantity, customer and address explain the purpose of the method and its parameters without extra comments.

> **Figure 2, page 5:** 02-order-facade.png, line 7. The method name describes the action; its three parameters describe the information the client provides.

2 Single Responsibility

OrderFacade controls the sequence of operations. Stock calculations, balance updates and shipment creation remain inside their own services.

> **Figure 2, page 5:** 02-order-facade.png, lines 14-17. The facade delegates work to the services. **Figure 4, page 7:** 04-payment.png, lines 12-21, shows that the balance check and deduction belong to PaymentProcessor.

3 Encapsulation and a Simple Interface

The facade stores its services in private fields. Main does not create or call them. It uses the simple facade API and receives a String instead of a subsystem object.

> **Figure 2, page 5:** 02-order-facade.png, lines 2-5, keeps service references private. **Figure 1, page 4:** 01-main.png, lines 3 and 16, shows that Main creates the facade and calls placeOrder without using a service directly.

4 Small Methods and Early Validation

The reserve method checks the quantity before changing the stock. Zero or negative quantities are rejected immediately, keeping the main logic simple and preventing invalid state changes.

> **Figure 3, page 6:** 03-inventory.png, lines 14-19. Validation rejects invalid quantities and insufficient stock. The stock changes only at line 21, after these checks.

5 Error Handling and Rollback

The paid flag records whether the charge succeeded. Money is refunded only after a successful charge. The stock reservation is released after either payment or shipping failure, and the original exception is passed back to the client.

> **Figure 2, page 5:** 02-order-facade.png, lines 12, 15 and 18-23. The flag tracks a completed payment; the catch block refunds it when necessary, releases the books and rethrows the error.

4 Demonstration and Testing

Initial data: 3 books; Alice has 3000 KZT and Bob has 500 KZT. Main runs five scenarios in sequence using the same facade instance. The table shows the state after each step.

Scenario	Result	Stock	Alice KZT
1 Alice orders 1 book	SHIP-1, charged 1000 KZT	2	2000
2 Bob orders 2 books	Insufficient funds; stock released	2	2000
3 Alice orders 2 books with no address	Invalid address; payment refunded and stock released	2	2000
4 Alice orders 2 books	SHIP-2, charged 2000 KZT	0	0
5 Alice orders 1 book	Out of stock	0	0

Automated Tests

All 10 tests in OrderFacadeTest passed. They cover a successful order, insufficient stock, declined payment, zero and negative quantities, an unknown customer, blank or null customer names, and blank or null addresses. In the rollback tests, a successful order for all 3 books after a failure confirms that stock and funds are available again.

The code was compiled with JDK 21 using --release 17 and executed on JDK 21. It uses only the standard library and targets Java 17.

5 Conclusion

The facade hides four services, their call order and rollback logic. Main uses one method instead of coordinating the whole process. The services do not reference OrderFacade. Their public methods remain available to advanced clients that want to use them directly.

This is a single-threaded educational simulation. Data is stored in memory and is lost when the program ends. There are no real payments, deliveries or messages. Shipping validates the address before storing a shipment. Notification only stores a String in memory; notification failure is not modeled.

6 GitHub Repository Link

https://github.com/bigstep832/assignment-4-facade-pattern

Figure 1. Main.java

Complete source, lines 1-22 | 01-main.png

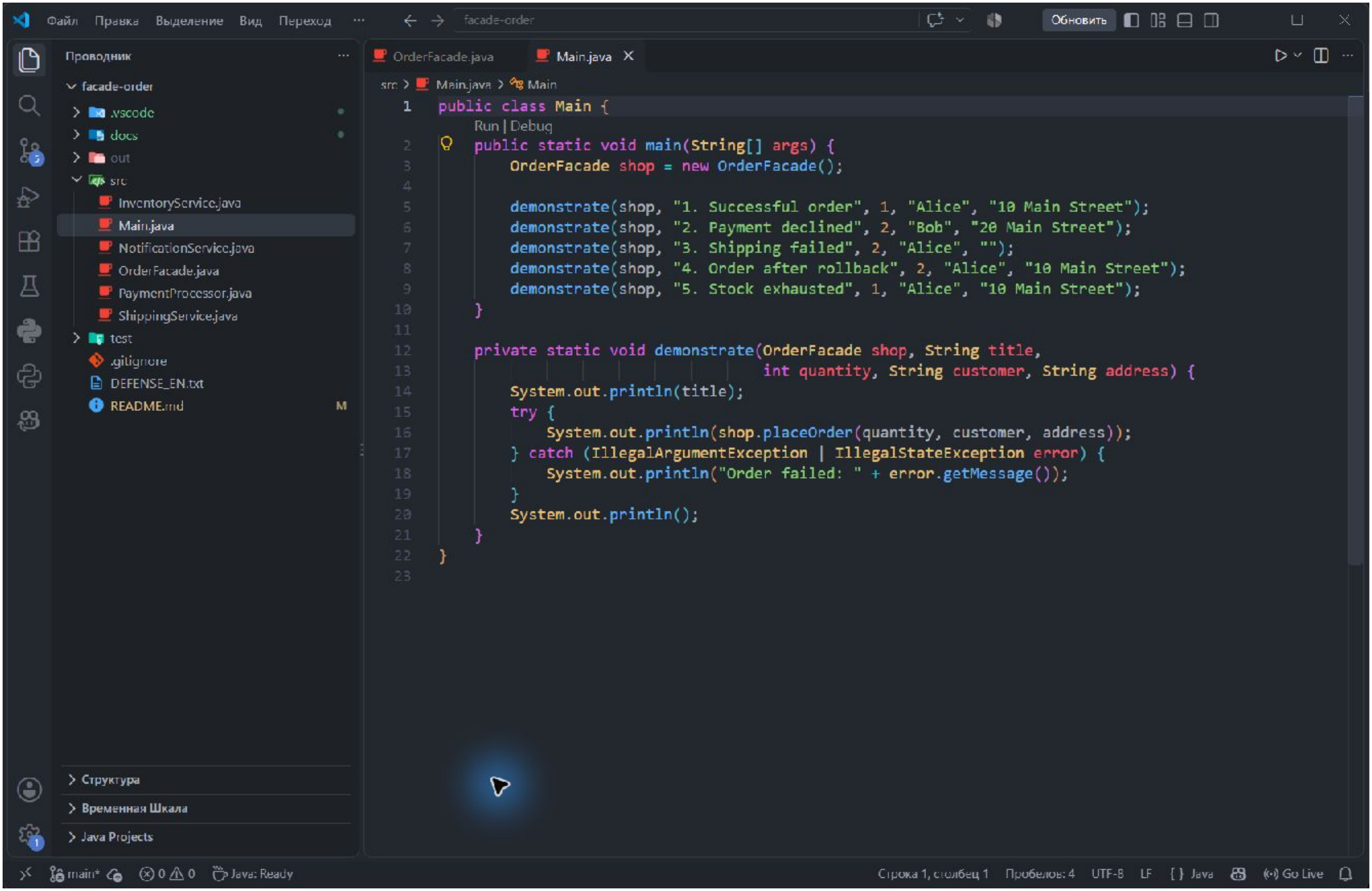


Figure 2. OrderFacade.java

Complete source, lines 1-26 | 02-order-facade.png

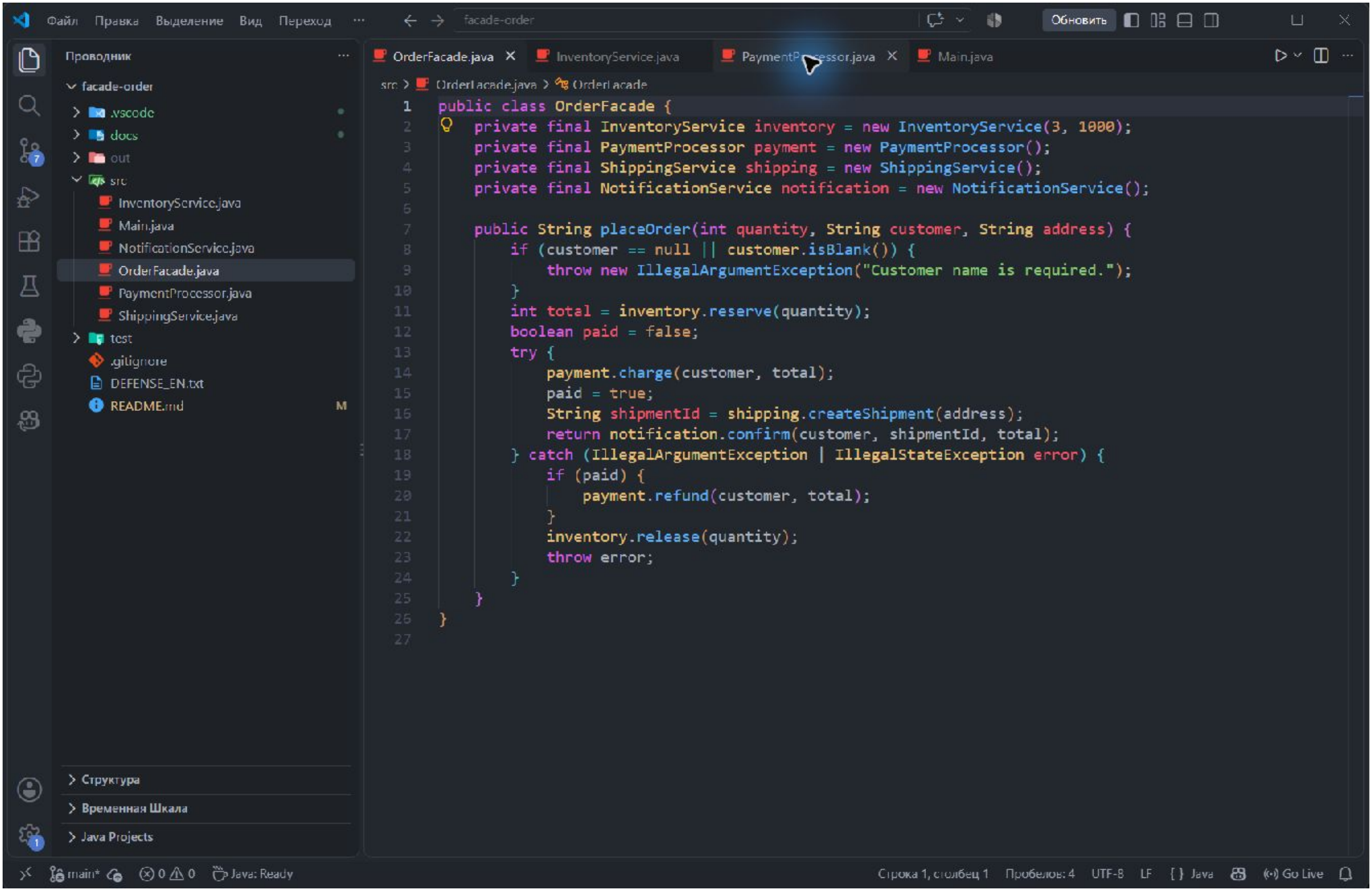


Figure 3. InventoryService.java

Complete source, lines 1-35 | 03-inventory.png

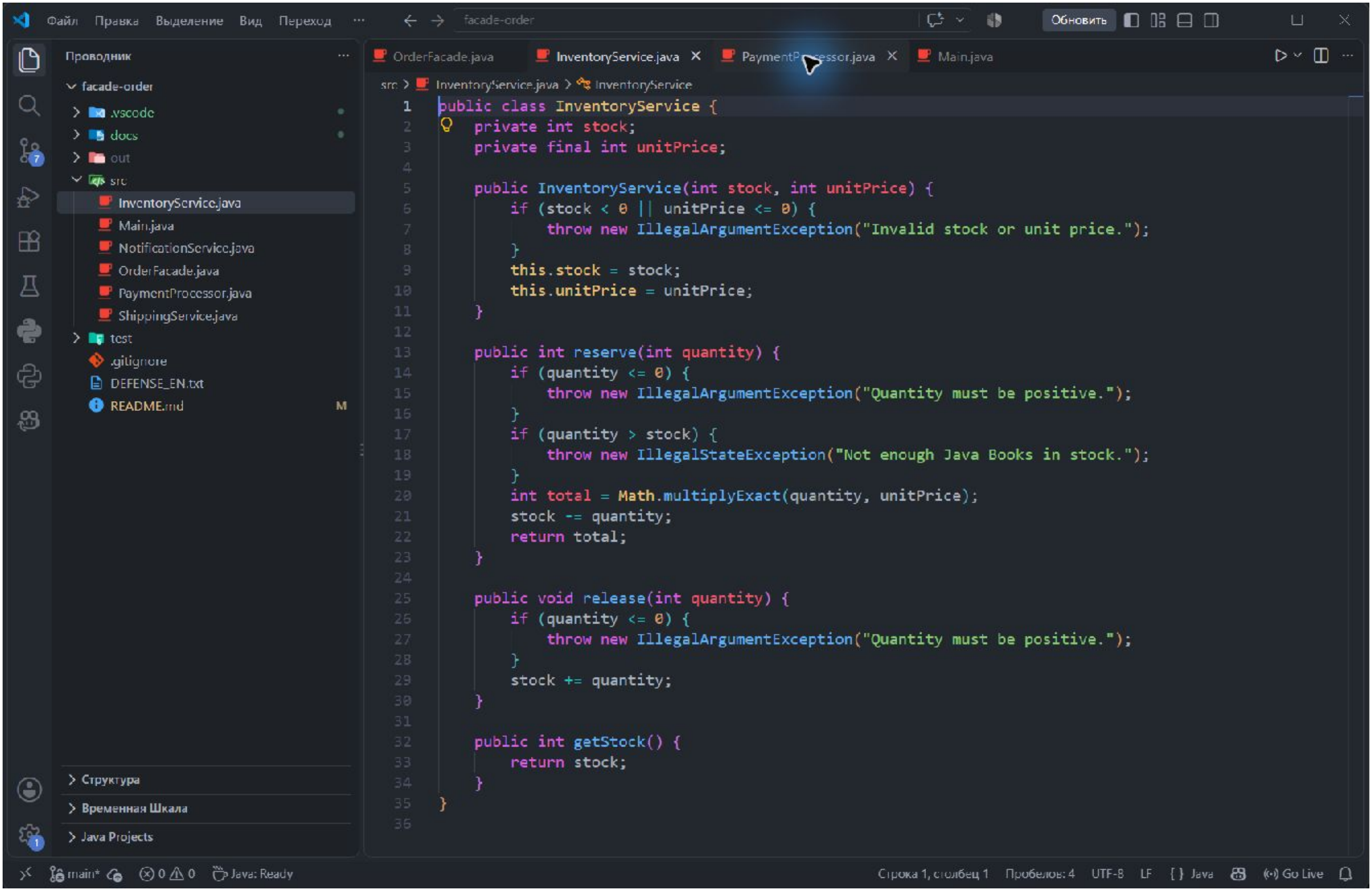


Figure 4. PaymentProcessor.java

Complete source, lines 1-36 | 04-payment.png

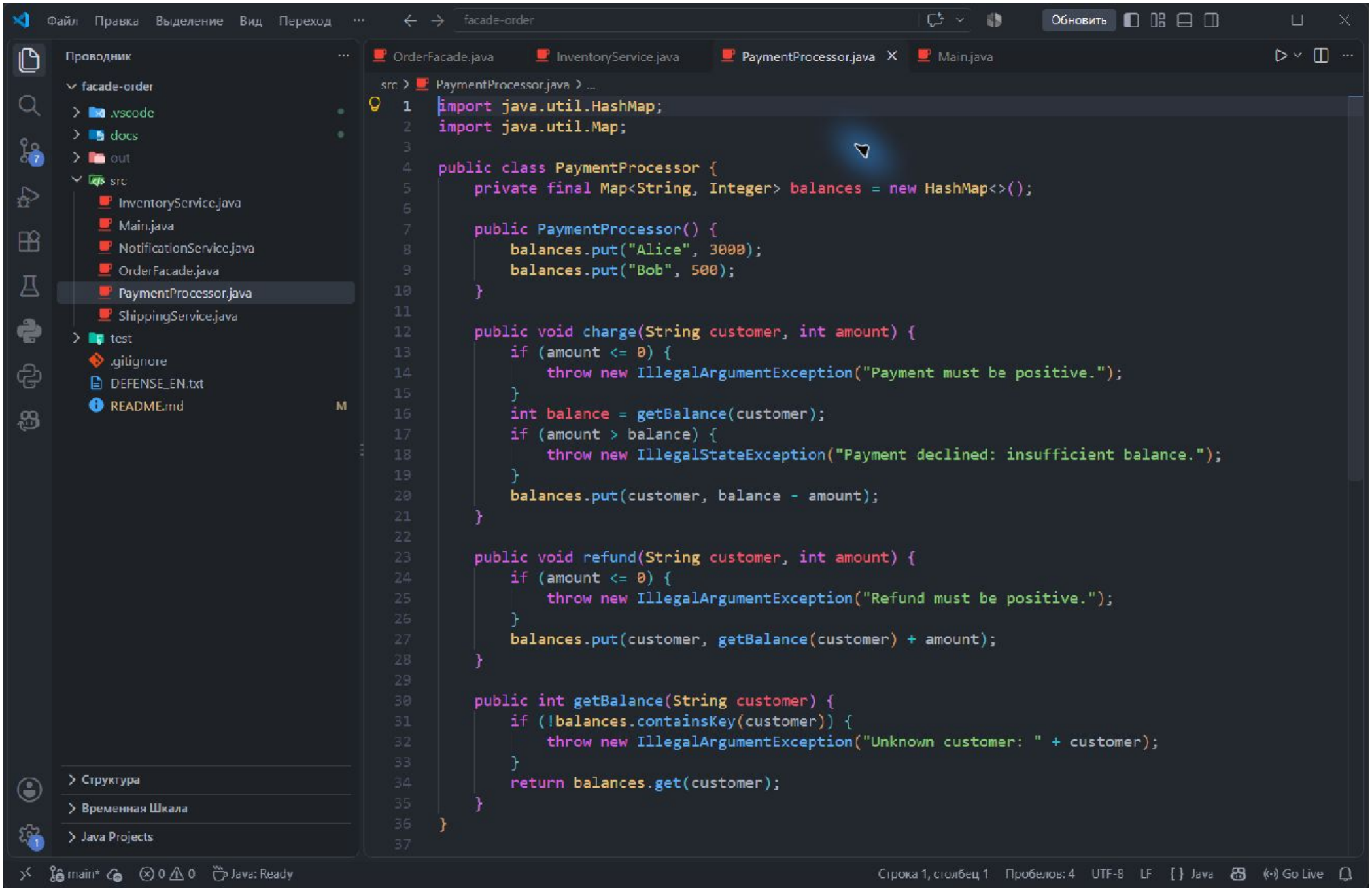


Figure 5. ShippingService.java

Complete source, lines 1-20 | 05-shipping.png

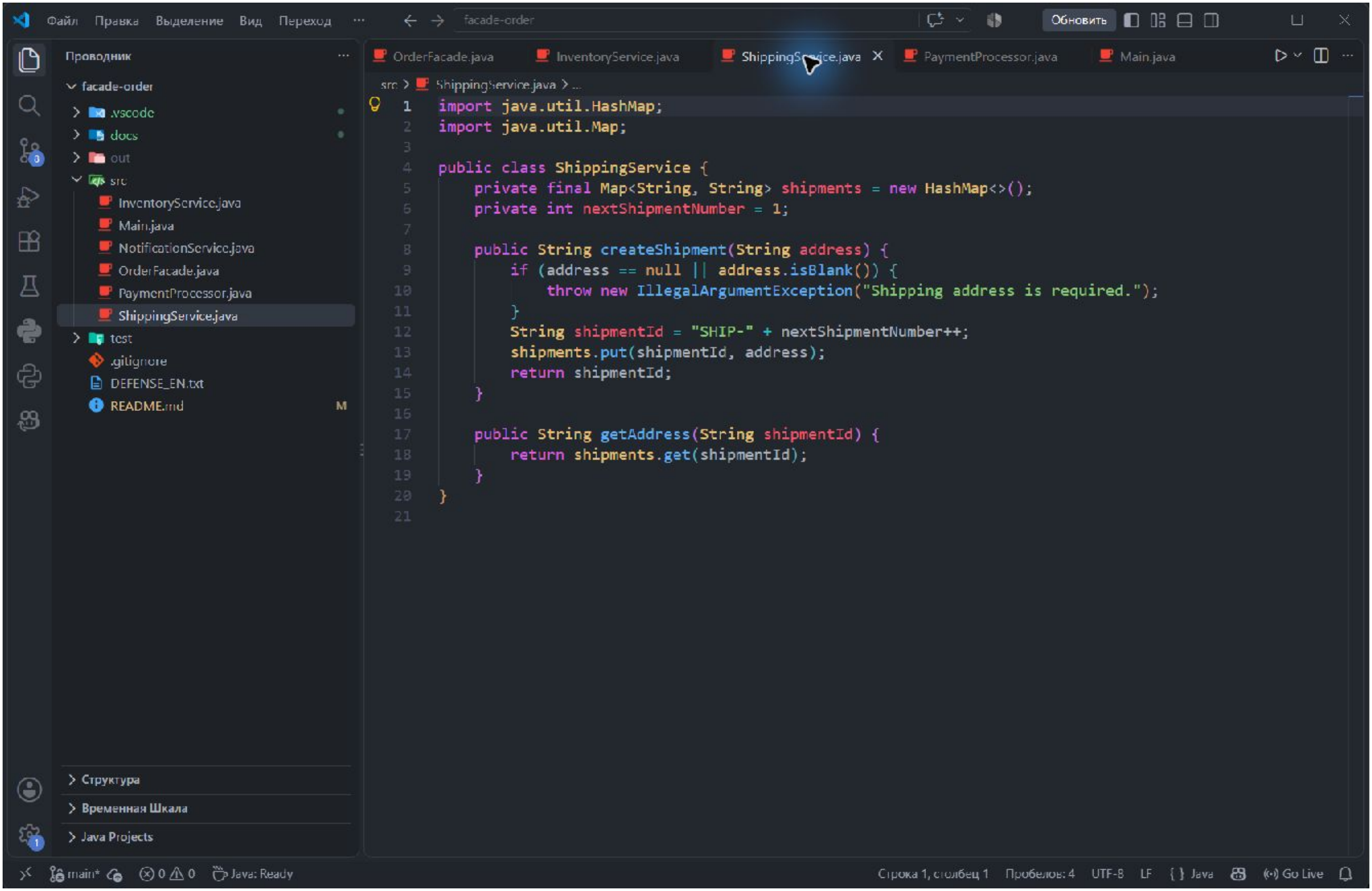


Figure 6. NotificationService.java

Complete source, lines 1-17 | 06-notification.png

